

# Computer Simulation

## Lesson 4: Functions

Takumi Miyoshi, Taku Yamazaki

Special thanks to: Dr. Pham Thi Huong

## Lesson 4

### Functions

- Introduction to m-files
- Scripts
- Functions
  - Input/output
  - Nested function
  - Functions of variable-length arguments
- Practices

Takumi Miyoshi / Taku Yamazaki

Lesson 4: Functions

## Introduction to m-file (1)

- Two ways to calculate in MATLAB
    - (1) **Directly enter commands** on Command Window
    - (2) **Run an m-file**
  - For a small and simple calculation, to perform individual operations → (1)
  - For a large program → (2) m-file
  - Write and save a MATLAB program in a file with extension .m, E.g., example1.m
- The kind of file is called as m-file

Takumi Miyoshi / Taku Yamazaki

Lesson 4: Functions

## Introduction to m-file (2)

- Allow reusing sequences of commands
- Two kinds of m-file
  - **Script file**: store commands exactly as you would type at the Command Window
    - a script never returns a value or accepts input arguments
  - **Function file**: more flexible and easily extensible

Takumi Miyoshi / Taku Yamazaki

Lesson 4: Functions

# Scripts

script.m

```
%Explanation  
Get the input  
Calculate the result  
Display the output
```

script1.m

```
% The script calculates the  
% area of a circle  
radius = 9  
area = pi * (radius^2)
```

- Display a script content: **type script\_name**
- Run a script by entering the script name (without the extension .m):, E.g.: **script1**

# When to use scripts

- When you want to repeat a set of commands in the Command Window many times but you don't want to type them many times
- You tell MATLAB what's supposed to happen, just like a script that says what's supposed to happen in a play or a movie, by writing a script

# Functions

- Introduction to Functions
  - Built-in functions
  - **User-defined function**
- Definition of Functions
- Input / output
- Nested functions
- Functions of variable-length arguments

# Introduction to Functions

- To calculate the area of another circle
  - Solution 1: using the script and manually update the values of radius in the script and rerun it each time
  - Solution 2: using the function
- Two kind of functions:
  - Built-in functions: sin, sqrt, sum, ...
  - **User defined functions**

# Definition of Functions

**function\_name.m**

```
function[output]=function_name(input)
%Explanation
Commands
end
```

- Note:
  - The declaration with the reserved word **function** must be the first line
  - Function's name should be the same as m-file's name
  - One or no output, can omit the square brackets []
  - No input, can omit the parentheses ()
  - Keyword “**end**” is optional

# Example of Functions

**calarea.m**

```
function area = calarea(radius)
%This function calculate the area of a circle
area = pi * radius.^2;
```

- Display the function content: **type** calarea
- Calling a function:  
radius = 5; calarea(radius)  
radius = [1:3]; calarea(radius)

## Input / Output (1)

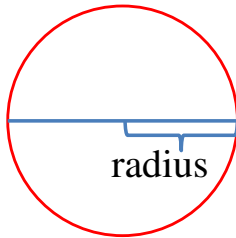
- Variables in a function are local variables
  - Stored in the function workspace
  - Used only inside function
- A function can has
  - Zero, one or multiple input arguments
  - Zero, one or multiple output arguments

## Input / Output (2)

- **function** func00
- **function** func10(in1)
- **function** funcN0(in1, in2, ..., inN)
- **function** out1=func01
- **function** out1=func11(in1)
- **function** out1=funcN1(in1, in2, ..., inN)
- **function** [out1, out2,..., outN] = func0N
- **function** [out1, out2,..., outN] = func1N(in1)
- **function** [out1, out2,..., outN] =  
funcNN(in1, in2, ..., inN)

## Example

- Write a function to calculate the area and the circumference of a circle
- $\text{Area} = \pi * \text{radius}^2$
- $\text{Circumference} = 2 * \pi * \text{radius}$



## 0 input argument, 0 output argument

```
function AreaAndCir00
radius = 5;
area = pi*radius^2; %Area
cir = 2*pi*radius; %Circumference
fprintf('Area = %5.2f, Circumference = %5.2f\n', area, cir);
end
```

Calling function:  
>>AreaAndCir00

## 1 input argument, 0 output argument

```
function AreaAndCir10(radius)
area = pi.*radius.^2; %Area
cir = 2.*pi.*radius; %Circumference
fprintf('Area = %5.2f, Circumference = %5.2f\n', area, cir);
end
```

Calling function:  
>>radius = 5;  
>>AreaAndCir10(radius)

## 0 input argument, 1 output argument

```
function out = AreaAndCir01
radius = 5;
area = pi*radius^2; %Area
cir = 2*pi*radius; %Circumference
out = [area cir];
end
```

Calling function:  
>>out = AreaAndCir01  
>>AreaAndCir01 →???

## 1 input argument, 1 output argument

```
function out = AreaAndCir11(radius)
area = pi.*radius.^2; %Area
cir = 2.*pi.*radius; %Circumference
out = [area cir];
end
```

Calling function:

```
>>radius = 5; % radius = [1 2 3]
>>out = AreaAndCir11(radius)
```

## 0 input argument, multiple output arguments

```
function [area cir] = AreaAndCir0N
radius =5;
area = pi*radius^2; %Area
cir = 2*pi*radius; %Circumference
end
```

Calling function:

```
>>[area circumference] = AreaAndCir0N
>>AreaAndCir0N →?????
```

## 1 input argument, multiple output arguments

```
function [area cir] = AreaAndCir1N(radius)
area = pi.*radius.^2; %Area
cir = 2.*pi.*radius; %Circumference
end
```

Calling function:

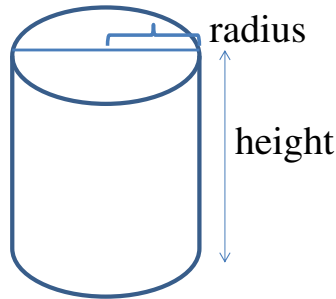
```
>>radius = 5; %radius =[1:3];
>>[area circumference] = AreaAndCir1N(radius)
```

## Nested functions

- Nested functions are functions contained within other functions
- **When functions are nested, every function must have an end statement**
- A variable defined in the outer function could be used in an inner function and vice versa
- If a variable is not used in the outer function the variable remains local to the nested function

## Nested functions: Example

- Calculate the volume of a cylinder
  - volume = area \* height;
  - area =  $\pi \times \text{radius}^2$  %surface area



## Nested functions: Answer

```
function volume = CylinVolNest(radius, height)
    % calculate the volume of a cylinder
    volume = area * height;
    % begin inner function base
    function outarea = area
        outarea = pi*radius^2;
    end % end inner function base
end % end outer function CylinVolNest
```

## Variable Numbers of Arguments

- A function can have the variable number of input and output arguments
- A built-in cell array
  - **varargin**: store the variable number of input arguments
  - **varargout**: store the variable number of output arguments
- The function **nargin** returns the number of input arguments that were passed to the function
- The function **nargout** determines how many output arguments are expected to be returned from a function

## Variable Numbers of Arguments: Example

- Write a script to calculate 1) the area and 2) the circumference of a circle when the radius is entered by users
- If a user enters 2 input arguments, the second argument will be treated as the height of the equivalent cylinder
  - Calculate 3) the volume of the cylinder

# Combination a function and a script

- Write a script to calculate the volume of a cylinder
  - A function **getinfor** : to ask a user for a radius and a height
  - A function **getarea** : to calculate the area of the circle
  - A function **getvolume** : to calculate the volume
    - $\text{Volume} = \text{area} * h$
    - Area is obtained by calling **getarea**
  - Display the result

# Practices

- 4.1. Write a function that will ask a user for a negative number; loop for error-check to make sure that the number is negative, and return the negative number
- 4.2. Write a function to convert temperature in deg C to deg F; and if there are 2 outputs are expected, convert to both degs F and K
  - $\text{degreesF} = 9/5 * \text{degreesC} + 32;$
  - $\text{degreesK} = \text{degreesC} + 273.15;$

Thank you very much for your attendance!